TP 3 — Formats de Données, REST/SOAP et GraphQL/gRPC

Ralph Bou Nader

Master — Ingénierie Informatique

Objectifs

Ce TP couvre trois chapitres du cours en un seul parcours pratique :

- **Chapitre 3** : manipuler JSON et XML, comprendre la négociation de contenu HTTP.
- **Chapitre 4** : concevoir des URLs RESTful, tester une vraie API REST, comparer avec un service SOAP réel.
- **Chapitre 5** : interroger une API GraphQL publique, concevoir un schéma GraphQL et un fichier `.proto` gRPC.

Prérequis

- Postman (déjà utilisé au TP1).
- Un navigateur web.
- Aucune installation supplémentaire n'est nécessaire : toutes les API utilisées sont publiques et gratuites.

Partie 1 — Formats de Données (Chapitre 3)

Tâche 1 — Identifier les types JSON dans une réponse réelle

1. Dans Postman, effectuez `GET https://jsonplaceholder.typicode.com/users/1`.
2. Observez le corps de la réponse.

Question à consigner : pour chacun des 6 types JSON vus en cours (string, number, boolean, null, object, array), trouvez un exemple dans cette réponse — ou indiquez que ce type est absent si c'est le cas.

Tâche 2 — Convertir du JSON en XML

Reprenez la réponse obtenue à la Tâche 1 (un objet utilisateur avec `name`, `email`, et une `address` imbriquée).

Travail à faire : transcrivez cet objet JSON en un document XML valide, en respectant les règles vues en cours :

- Un unique nœud racine (par exemple `<user>`).
- Un choix cohérent entre éléments enfants et attributs pour chaque champ.
- La sous-structure `address` correctement imbriquée.

Question à consigner : pour le champ `id`, avez-vous choisi de le représenter comme un attribut (`<user id="1">`) ou comme un élément (`<id>1</id>`) ? Justifiez avec le critère vu en cours.

Tâche 3 — Négociation de contenu et codes d'erreur

1. Dans Postman, effectuez `GET https://httpbin.org/status/406`. Observez le code retourné.
2. Effectuez `GET https://httpbin.org/status/415`. Observez le code retourné.
3. Effectuez `POST https://httpbin.org/anything` avec l'en-tête `Content-Type: application/xml` et un corps XML de votre choix. Observez, dans la réponse JSON renvoyée par httpbin, la section `"headers"` : votre `Content-Type` y apparaît tel quel.

Question à consigner : httpbin.org ne rejette jamais vos requêtes, quel que soit le format envoyé — répondez, en une phrase, à quoi correspondrait chacun des deux codes (406, 415) si ce serveur pratiquait une véritable négociation de contenu comme vu en cours.

Partie 2 — REST et SOAP (Chapitre 4)

Tâche 4 — Concevoir des URLs RESTful

Scénario : vous concevez l'API REST d'une bibliothèque en ligne, avec des livres, des auteurs, et des emprunts.

Travail à faire : proposez, pour chacune des actions suivantes, la méthode HTTP et l'URL RESTful correspondante, en appliquant les principes du Chapitre 4 (ressources au pluriel, pas de verbe dans l'URL, chaîne de requête pour le filtrage) :

1. Lister tous les livres.
2. Récupérer le livre d'identifiant 42.
3. Lister uniquement les livres disponibles (non empruntés).
4. Emprunter le livre 42 (créer un emprunt).
5. Récupérer tous les livres d'un auteur précis (identifiant 7).
6. Mettre fin à l'emprunt d'identifiant 15.

Tâche 5 — Tester la non-idempotence de POST

1. Dans Postman, effectuez deux fois de suite `POST https://jsonplaceholder.typicode.com/posts` avec exactement le même corps JSON.
2. Comparez les deux réponses.

Question à consigner : les deux réponses sont-elles identiques ? Que cela démontre-t-il concrètement à propos de l'idempotence de POST, vue en cours ?

Tâche 6 — Interroger un vrai service SOAP

Nous allons utiliser un service SOAP public de calculatrice, couramment utilisé à des fins pédagogiques.

1. Créez une nouvelle requête Postman, méthode `POST`, URL `http://www.dneonline.com/calculator.asmx`.
2. Onglet **Headers**, ajoutez : `Content-Type: text/xml; charset=utf-8`.
3. Onglet **Body** → `raw` → `XML`, avec le contenu suivant :

```xml
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
               xmlns:xsd="http://www.w3.org/2001/XMLSchema"
               xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add xmlns="http://tempuri.org/">
      <intA>4</intA>
      <intB>7</intB>
    </Add>
  </soap:Body>
</soap:Envelope>
```

4. Envoyez la requête et observez la réponse (une enveloppe SOAP contenant le résultat).

Questions à consigner :

- Identifiez, dans la réponse, l'`Envelope` et le `Body` vus en cours.
- Cette requête utilise-t-elle un verbe HTTP différent selon l'opération demandée (`Add`, `Subtract`…) ? Comparez avec l'approche REST de la Tâche 4.

- Le nom de l'opération (`Add`) apparaît-il dans l'URL, ou ailleurs ? Qu'est-ce que cela confirme à propos de la différence structurelle entre REST et SOAP ?

Partie 3 — GraphQL et gRPC (Chapitre 5)

Tâche 7 — Écrire une requête GraphQL contre une vraie API

Nous utilisons l'API GraphQL publique **Rick and Morty**, qui dispose d'un explorateur interactif.

1. Ouvrez `https://rickandmortyapi.com/graphql` dans votre navigateur.
2. Dans l'éditeur, écrivez et exécutez la requête suivante :

```
query {
  character(id: 1) {
    name
    status
    species
    episode {
      name
      air_date
    }
  }
}
```

3. Observez la réponse JSON obtenue.

Question à consigner : cette requête a permis de récupérer, en un seul appel, le personnage **et** la liste de ses épisodes. Avec une API REST classique, combien d'appels auriez-vous probablement dû faire pour obtenir la même information ? Reliez votre réponse à la notion d'*under-fetching* vue en cours.

Tâche 8 — Modifier la requête pour observer l'over-fetching

1. Dans le même éditeur, exécutez cette fois :

```
query {
  character(id: 1) {
    name
  }
}
```

Question à consigner : comparez la taille des deux réponses obtenues (Tâche 7 vs Tâche 8). En quoi cette différence illustre-t-elle concrètement comment GraphQL évite l'*over-fetching* ?

Tâche 9 — Concevoir un schéma GraphQL (exercice écrit)

Scénario : reprenez la bibliothèque en ligne de la Tâche 4.

Travail à faire : écrivez un schéma GraphQL (types + une query) permettant de récupérer un livre avec son auteur et la liste de ses emprunts en cours, sur le modèle vu en cours (Section 3.2 du Chapitre 5).

Tâche 10 — Concevoir un service gRPC (exercice écrit)

Scénario : toujours pour la bibliothèque en ligne, vous devez maintenant exposer, en interne, un service gRPC permettant de récupérer les informations d'un livre à partir de son identifiant.

Travail à faire :

1. Écrivez un fichier `.proto` définissant : un message `LivreRequest` (contenant un identifiant), un message `LivreResponse` (contenant titre, auteur, disponibilité), et un service `LivreService` exposant une méthode `GetLivre`.
2. Parmi les quatre modes de streaming vus en cours (unaire, server streaming, client streaming, bidirectionnel), lequel choisiriez-vous pour cette méthode `GetLivre` ? Justifiez.
3. Un second besoin apparaît : notifier en continu un tableau de bord interne de chaque nouvel emprunt, dès qu'il se produit. Quel mode de streaming choisiriez-vous cette fois ? Justifiez, en le comparant à votre réponse précédente.

Rédaction du rapport

Votre rapport doit inclure :

1. Pour chaque tâche pratique (1, 2, 3, 5, 6, 7, 8) : une capture d'écran de la requête et de la réponse.
2. Le document XML complet de la Tâche 2.
3. Les réponses rédigées à toutes les questions « à consigner ».
4. Le tableau des 6 URLs RESTful de la Tâche 4.
5. Le schéma GraphQL de la Tâche 9 et le fichier `.proto` de la Tâche 10, avec vos justifications.

Soumission

Déposez, sur la plateforme du cours, votre rapport au format PDF, incluant l'ensemble des éléments listés ci-dessus.

Barème indicatif

Critère	Points
Partie 1 — Formats de données (Tâches 1 à 3)	5
Partie 2 — REST et SOAP (Tâches 4 à 6)	7
Partie 3 — GraphQL et gRPC (Tâches 7 à 10)	6
Qualité et clarté du rapport	2
Total	**20**